- Cos'è il k-means?
  - E' un algoritmo di classificazione usato per classificare quando sappiamo che abbiamo k classi di output
  - Funziona su ellissi, dove cerchiamo di calcolare il entro di ellissi, e poi lo spostiamo Problema:
  - Output diversi per input uguali, siccome iniziamo casualmente. Facciamo linee di kmeans
  - Non possiamo gestire le non sfere
- Cos'è il DBScan?
  - E' un sistema di clustering basato sull'idea di prendere valori che sono vicini tra di sè
  - Qui il numero di cluster li decide l'algoritmo
  - Si creano 3 tipi di nodi:
    - Nodi core
    - Nodi bordi
    - Nodi disturbo
- Cos'è RDF
  E' un sistema per rapppresentare la conoscenza, in particolare è un datamodel per condividere dati sul web
  Ed è basato su:
  - Triple con <entità, tipo, valore>, sono le entità di base
  - Ogni tripla è organizzata come <soggetto, predicato, oggetto> anche rappresentabile come arco diretto
    - Oggetto può essere iri, blank node o literale
    - Predicato solo un iri
    - Soggetto solo iri e blank node
  - Iri, è un UUID per internet
- Cos'è RDFS
  E' un estensione di RDF usata per descrivere o modificare modelli dati
  abbiamo stili diversi, e relazioni con nomi diversi
  possiamo:
  - Creare classi con la relazione classOf
  - Creare tipo con type
  - Dire che qualcosa possiede una proprietà e tutto ciò che è correlato, subpropertyof
  - Inferenze indirette, tipo: :Mario :ama :Lucia
    - :ama :domain :Persona
      Allora il soggetto di ama è una persona

        - :ama :range :Persona
          Allora l'oggetto di ama è una persona
- Cos'è SPARQL
  E' un linguaggio per interrogare grafi rdf, le query descrivono gradi rdf e fanno pattern matching
  Fatto da:
    - Prefix iniziali per rendere più breve
    - select (qualcosa da selezionare)
        - Opzione di rendere DISTINCT ciò da selezionare
        - Opzione di fare il COUNT
        - Opzione di fare l'or attraverso UNION
        - Possiamo aggiungere group by
        - Dopo il group by possiamo fare condizioni con HAVING(count(...))
- Quali sono le assunzioni di OWL
    - Ciò che non è dimostrato allora è sconosciuto
    - Nomi diversi non implica entità diverse
    - Le proprietà possono avere queste caratteristiche:
        - Simmetriche, A=>B allora B=>A
        - Anti simmetrica, se A=>B e B=>A, allora B=A
        - Inverse of se io sono figlio di antonella, allora antonella è madre mia
        - Transitività, se A=>B e B=>C, allora A=>C
        - Functional: posso avere solo 1 arco uscente da me, qui si crea un reticolo
        - Inverse functional: posso avere solo 1 arco entrante
    - La base di conoscenza è divisa in:
        - Tbox = Relazioni tra le classi, quindi subclass, equivalentclass, disgiunzione, forall, cardinalità (restrizioni)
        - Abox = Dati delle classi, quindi type, ci rappresenta gli assiomi, quindi io amo caia, oppure io sono persona
          Un dato è nella base di conoscenza solo se soddisfa sia Tbox che Abox
- Breve calcolo relativo al cnn
- Cos'è una RNN?
    - E' un percettore di cui output viene sia dato, ma poi dato ad input a se stesso
    - Viene usato per elaborare sequenze di dati sequenziali
    - Nota: siccome si porta a se stesso, i pesi sono gli stessi di 1 percettore tranne per w2 he è il peso per se stesso
    - Nota: più colleghiamo il percettore a se stesso, e più diventa difficile allenarsi

        - Il gradiente che esplode o sparisce
          Soluzione: LSTM
- Deep learning e vantaggi rispetto ad algoritmi ad apprendimento superficiale
    - Sono più percettori messi insieme, quindi permettono di classificare o generare robe più complesse
    - L'idea è creare più layer di percettori, così ogni layer impara qualcosa di diverso. Spesso i primi sono generici, e poi si approfondiscono
        - Se possibile si fanno percettori probabilistici così da rendere più facile la modifica
        - Questo però rende più difficile il calcolo del gradiente, siccome ci deve essere la backpropagazione
          attraverso un algoritmo chiamato minibatch gradiant descent
          Però questo crea altri problemi, e da qui si sono creati algoritmi per poter decidere come cambiare efficentemente il gradiente
- Cos'è il transfer learning?
    - Si basa sull'idea che, una rete neurale i layer più bassi sono spessi i più generici
      E quindi si usano reti già allenate sui layer più bassi per portarsi avanti
- Quali sono le problematiche relative al utilizzo delle reti neurali? quali algoritmi sono stati proposti per sopperire a tali problematiche?
- Cos'è overfitting e come si gestisce
    - E' quando una ML si abitua troppo alla training set
    - Si gestisce o allenandolo di meno (early stopping)
    - dividendo training e test set (lo rivela)
    - oppure a seconda dell'implementazione fare delle operazione. Es. decision tree tagliamo parti dell'albero, oppure non permettiamo di andare troppo specifici
    - Ridurre la complessità del modello
    - Applicare tecniche di regolarizzazione
    - Capitato nelle domande: fare un esempio nei problemi di classificazione
- PDDL e STRIPS
  PDDL e STRIPS sono 2 formalismi per potere rappresentare le informazioni per poter essere risolti da un algoritmo di pianificazione
    - STRIPS è più hardcoded, dobbiamo scrivere manualmente tutte le informazioni
      Tipo:
        - Wall_0,0 => muro 0,0
        - Pacman_0,0_0 => pacman nel tempo 0 è in posizione 0,0
          Problema: più diventa complesso il problema, e più esso tenderà ad esplodere
    - PDDL, è un pò più programmato di STRIPS
      Tutto questo si basa nel definire dei domin/problemi, e poi dare programmaticalmente delle soluzioni

        - Domini: le regole del mondo, le azioni
        - Problemi: i dati del mondo
          E qui le azioni vengono definite da variabili, così da non dovere ripetere troppe volte
- I problemi dell'apprendimento di una rete neurale e le soluzioni algoritmiche
    - Overfitting
    - Underfitting
    - Vanishing gradiant descent
        - Normalizza
        - Batch normalization
    - Exploding gradient descent
        - Normalizza
    - Punti minimi locali
        - usare algoritmi che permettono di più l'esplorazione, tipo momentum o adam
- La funzione di distanza nell'intelligenza artificiale subsimbolica
  La funzione di distanza è una funzione che calcola quanto 2 valori sono simili tra di loro
  Spesso un esempio di funzione di distanza è la funzione euclidea, che calcola la distanza cartesiana, il problema arriva
  quando non abbiamo delle coordinati ma dei valori letterali a cui dobbiamo in qualche modo attribuire i valori
- Limiti e vantaggi A
  Il A è un algoritmo di ricerca.
  Essa è molto veloce, e riesce ad ottenere il percorso con costo migliore attraverso le euristiche.
  Quando noi siamo in un grafo orientato, noi abbiamo 2 valori g(p), h(p, x).
  Per ogni nodo esplorato, noi abbiamo g(p), e calcoliamo g(p) + h(p', x) per ogni nodo raggiungibile
  E poi scegliamo di esplorare quello con il valore minore
  Problemi:
    - Trovare una funzione euristica non è sempre facile, spesso si usa la distanza però non è sempre ottimale
        - Se l'euristica è fatta male, sarà simile all'algoritmo di djstrak
    - E' costoso siccome dobbiamo tenere in memoria tutti i nodi esplorati
- Cos'è un algoritmo di ricerca
  Un algoritmo di ricerca è un modo per risolvere problemi di cui conosciamo tutti i possibili stati, ed il nostro obiettivo finale
  E l'obiettivo è, dato uno stato iniziale, una sequenza di azioni che ci porta al nostro obiettivo
  E se possibile, raggiungere una soluzione ottimale, cioè quella col costo minore

Nota: siccome esploriamo i nostri stati, è possibile raggiungere un posto dove il nostro spazio di stati esplode, cioè diventa troppo grande

- Gradiente cosa rappresenta ed applicazioni
  - Rappresenta quanto una varaiazione del peso da all'output
  - Ci da la direzione che da alla nostra funzione
- Cos'è il percettore?
  - E' un qualcosa per cercare di fare la classificazione su più punti nel piano
  - Nota: il numero di features indica la complessità, questo vuol dire che ottimalmente dovremmo già sapere più o meno come sarà il grafo
- Cos'è un decision tree?
  - E' un algoritmo di classificazione divide et impera
  - Funziona tutto nel calcolare quanto gli attributi sono impuri
    - Più sono impuri, e più contiene elementi di più classi
      Noi vogliamo minimizzare l'impurità, mettendo come come foglie ciò che è meno puro
    - Però se troppo puro no siccome overfitting
    - Se un ramo è puro, vuol dire che tutte le classi sono le stesse, quindi nuova foglia
    - Se non abbiamo nessun attributo, allora ci mettiamo la classe più frequente
    - Se invece tutto è troppo impuro, dobbiamo dividere ancora di più
- Quali sono le caratteristiche di un ambiente della sub-simbolica?
  - Accessibile vs Non accessibile (se abbiamo tutto l'ambiente)
  - Deterministico vs Non
  - Statico vs Discreto (sistema parallelo vs asincrono)
  - Continuo vs Discreto (azioni finite o no)
  - Episodici vs Sequenziali (le nostre azioni passate hanno importanza)
- Differenze spaziali e temporali degli algoritmi BFS e DFS, spiegare la ricerca iterativa e come questa sia una possibile soluzione ai problemi dei due algoritmi
- In RDFS è possibile applicare i vincoli di cardinalità? In OWL invece? fare un esempio utilizzando un linguaggio a piacere
- Mdp come funziona, come approcciarsi a robe infinite, reward positivi e negativi.
- Problemi ML:
  - Non abbastanza dati
  - Falsi positivi
- Come valutare un modello?
  - Accuracia, che da qui si crea precision (falsi positivi) e recall (falsi negativi)
- Come si allena un ML?
  - Batch learning, dai tutto in 1 botta

- Sequential learning, dai un pò alla volta